

Manejo de Múltiples Archivos en C

Programación en C

16 de febrero de 2026

1. Escribir múltiples archivos — Registro de ventas diarias

Caso de uso: Guardar ventas de cada día del mes en archivos separados.

Listing 1: Escritura de ventas diarias

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char nombre[30];
6     float ventas[] = {1500.50, 2300.75, 1800.00};
7
8     for (int dia = 1; dia <= 3; dia++) {
9         sprintf(nombre, "venta_dia_%d.txt", dia);
10        archivo = fopen(nombre, "w");
11        fprintf(archivo, "%.2f\n", ventas[dia-1]);
12        fclose(archivo);
13    }
14
15    printf("3 archivos de ventas creados\n");
16    return 0;
17 }
```

Resultado: Crea venta_dia.1.txt, venta_dia.2.txt, venta_dia.3.txt

Compilación / Salida:

	gmon	15/02/2026 20:37	Archivo OUT
	venta_dia_1	15/02/2026 20:37	Documento de tex...
	venta_dia_2	15/02/2026 20:37	Documento de tex...
	venta_dia_3	15/02/2026 20:37	Documento de tex...
	Project1	15/02/2026 20:37	Aplicación
	main	15/02/2026 20:37	C Source File
	main.o	15/02/2026 20:37	Archivo O
	Makefile.win	15/02/2026 20:37	Archivo WIN
	Project1	15/02/2026 20:36	Dev-C++ Project F...

2. Leer múltiples archivos — Calcular total de ventas

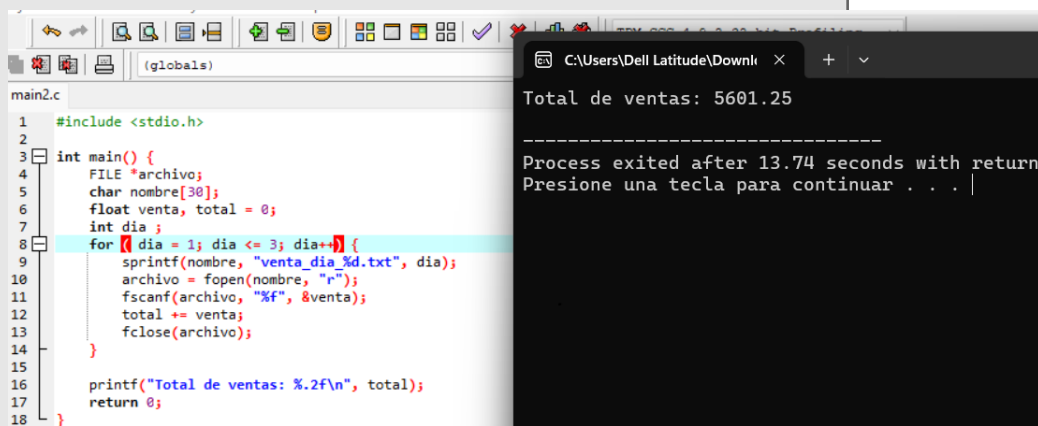
Caso de uso: Sumar ventas de todos los días.

Listing 2: Lectura y suma de ventas

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char nombre[30];
6     float venta, total = 0;
7
8     for (int dia = 1; dia <= 3; dia++) {
9         sprintf(nombre, "venta_dia_%d.txt", dia);
10        archivo = fopen(nombre, "r");
11        fscanf(archivo, "%f", &venta);
12        total += venta;
13        fclose(archivo);
14    }
15
16    printf("Total de ventas: %.2f\n", total);
17    return 0;
18 }
```

Salida: Total de ventas: 5600.25

Compilación / Salida:



The screenshot shows a code editor window with the C program code from Listing 2. The code is highlighted in a light blue color. To the right of the code editor is a terminal window. The terminal window displays the output of the program: "Total de ventas: 5601.25". Below this output, the terminal shows the message "Process exited after 13.74 seconds with return" and "Presione una tecla para continuar . . . |".

3. Escribir datos de sensores — Temperatura por hora

Caso de uso: Guardar lecturas de temperatura cada hora.

Listing 3: Escritura de temperaturas por hora

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char nombre[30];
6     float temperaturas[] = {18.5, 19.2, 20.1, 21.5};
7
8     for (int hora = 0; hora < 4; hora++) {
9         sprintf(nombre, "temp_hora_%d.txt", hora);
10        archivo = fopen(nombre, "w");
11        fprintf(archivo, "Hora %d: %.1fC\n", hora, temperaturas
12                [hora]);
13        fclose(archivo);
14    }
15
16    printf("Temperaturas guardadas\n");
17    return 0;
18 }
```

Compilación / Salida:

	temp_hora_0	15/02/2026 20:48	Documento de tex...
	temp_hora_1	15/02/2026 20:48	Documento de tex...
	temp_hora_2	15/02/2026 20:48	Documento de tex...
	temp_hora_3	15/02/2026 20:48	Documento de tex...

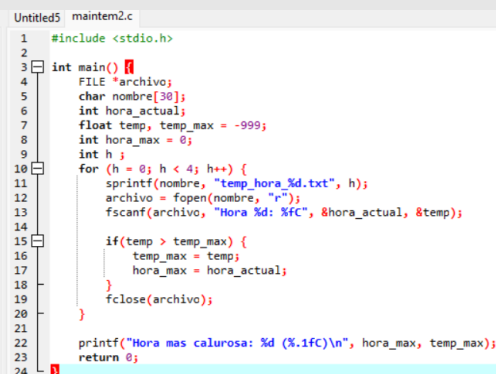
4. Leer y procesar — Encontrar temperatura máxima

Caso de uso: Buscar la hora más calurosa del día.

Listing 4: Búsqueda de temperatura máxima

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char nombre[30];
6     int hora_actual;
7     float temp, temp_max = -999;
8     int hora_max = 0;
9
10    for (int h = 0; h < 4; h++) {
11        sprintf(nombre, "temp_hora_%d.txt", h);
12        archivo = fopen(nombre, "r");
13        fscanf(archivo, "Hora %d: %fC", &hora_actual, &temp);
14
15        if (temp > temp_max) {
16            temp_max = temp;
17            hora_max = hora_actual;
18        }
19        fclose(archivo);
20    }
21
22    printf("Hora mas calurosa: %d (%.1fC)\n", hora_max,
23          temp_max);
24    return 0;
}
```

Compilación / Salida:



```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char nombre[30];
6     int hora_actual;
7     float temp, temp_max = -999;
8     int hora_max = 0;
9     int h;
10    for (h = 0; h < 4; h++) {
11        sprintf(nombre, "temp_hora_%d.txt", h);
12        archivo = fopen(nombre, "r");
13        fscanf(archivo, "Hora %d: %fC", &hora_actual, &temp);
14
15        if (temp > temp_max) {
16            temp_max = temp;
17            hora_max = hora_actual;
18        }
19        fclose(archivo);
20    }
21
22    printf("Hora mas calurosa: %d (%.1fC)\n", hora_max, temp_max);
23    return 0;
}
```

Hora mas calurosa: 3 (21.5C)

Process exited after 13.67 seconds with return value 0
Presione una tecla para continuar . . . |

5. Escribir logs de errores — Sistema de monitoreo

Caso de uso: Registrar errores en archivos por tipo.

Listing 5: Sistema de logs por tipo de error

```
1 #include <stdio.h>
2
3 void registrar_error(char *tipo, char *mensaje) {
4     FILE *archivo;
5     char nombre[40];
6     sprintf(nombre, "log_%s.txt", tipo);
7     archivo = fopen(nombre, "a"); /* append */
8     fprintf(archivo, "%s\n", mensaje);
9     fclose(archivo);
10 }
11
12 int main() {
13     registrar_error("red", "Conexion perdida");
14     registrar_error("disco", "Espacio insuficiente");
15     registrar_error("red", "Timeout en servidor");
16
17     printf("Errores registrados\n");
18     return 0;
19 }
```

Compilación / Salida:

 eje5	15/02/2026 20:55	Dev-C++ Project F...
 eje5	15/02/2026 20:56	Aplicación
 gmon	15/02/2026 20:56	Archivo OUT
 log_disco	15/02/2026 20:56	Documento de tex...
 log_red	15/02/2026 20:56	Documento de tex...

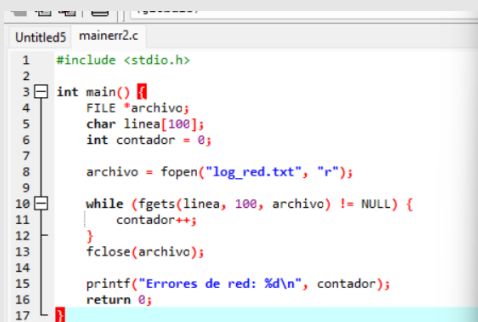
6. Leer y contar — Estadísticas de errores

Caso de uso: Contar líneas en archivo de log.

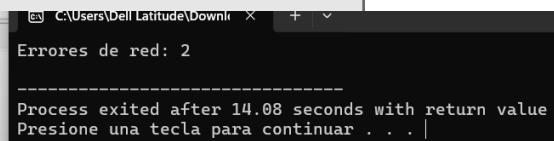
Listing 6: Conteo de errores en log

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char linea[100];
6     int contador = 0;
7
8     archivo = fopen("log_red.txt", "r");
9
10    while (fgets(linea, 100, archivo) != NULL) {
11        contador++;
12    }
13    fclose(archivo);
14
15    printf("Errores de red: %d\n", contador);
16    return 0;
17 }
```

Compilación / Salida:



```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char linea[100];
6     int contador = 0;
7
8     archivo = fopen("log_red.txt", "r");
9
10    while (fgets(linea, 100, archivo) != NULL) {
11        contador++;
12    }
13    fclose(archivo);
14
15    printf("Errores de red: %d\n", contador);
16    return 0;
17 }
```



```
Errores de red: 2
-----
Process exited after 14.08 seconds with return value
Presione una tecla para continuar . . . |
```

7. Copiar entre archivos — Backup de datos

Caso de uso: Hacer respaldo de archivo importante.

Listing 7: Copia de archivo para backup

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *origen, *destino;
5     char c;
6
7     origen = fopen("datos.txt", "r");
8     destino = fopen("datos_backup.txt", "w");
9
10    while ((c = fgetc(origen)) != EOF) {
11        fputc(c, destino);
12    }
13
14    fclose(origen);
15    fclose(destino);
16    printf("Backup completado\n");
17    return 0;
18 }
```

Compilación / Salida:

	datos_backup	15/02/2026 21:03	Documento de tex...
	gmon	15/02/2026 21:03	Archivo OUT
	eje7	15/02/2026 21:03	Aplicación
	mainback.o	15/02/2026 21:03	Archivo O
	mainback	15/02/2026 21:03	C Source File
	Makefile.win	15/02/2026 21:03	Archivo WIN
	main	15/02/2026 21:02	C Source File
	eje7	15/02/2026 21:02	Dev-C++ Project F...

8. Filtrar datos — Extraer valores altos

Caso de uso: Guardar solo temperaturas mayores a 20°C.

Listing 8: Filtrado de temperaturas altas

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *entrada, *salida;
5     float temp;
6
7     entrada = fopen("todas_temps.txt", "r");
8     salida = fopen("temps_altas.txt", "w");
9
10    while (fscanf(entrada, "%f", &temp) == 1) {
11        if (temp > 20.0) {
12            fprintf(salida, "%.1f\n", temp);
13        }
14    }
15
16    fclose(entrada);
17    fclose(salida);
18    printf("Filtrado completado\n");
19    return 0;
20 }
```

Compilación / Salida:

 gmon	15/02/2026 21:07	Archivo OL
 temps_altas	15/02/2026 21:07	Document
 eje8	15/02/2026 21:07	Aplicación

9. Consolidar archivos — Unir reportes

Caso de uso: Juntar reportes de 3 sucursales en uno solo.

Listing 9: Consolidación de reportes de sucursales

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *entrada, *salida;
5     char nombre[30], linea[100];
6
7     salida = fopen("reporte_total.txt", "w");
8
9     for (int sucursal = 1; sucursal <= 3; sucursal++) {
10        sprintf(nombre, "sucursal_%d.txt", sucursal);
11        entrada = fopen(nombre, "r");
12
13        while (fgets(linea, 100, entrada) != NULL) {
14            fputs(linea, salida);
15        }
16        fclose(entrada);
17    }
18
19    fclose(salida);
20    printf("Reportes consolidados\n");
21    return 0;
22 }
```

Compilación / Salida:

	gmon	15/02/2026 21:11	Archivo OUT
	reporte_total	15/02/2026 21:11	Documento de tex...
	eje9	15/02/2026 21:10	Aplicación

10. Procesar CSV — Análisis de calificaciones

Caso de uso: Leer notas y calcular promedio por alumno.

Listing 10: Cálculo de promedios desde archivo CSV

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char nombre[50];
6     float nota1, nota2, nota3, promedio;
7
8     archivo = fopen("notas.csv", "r");
9
10    /* Saltar encabezado */
11    fscanf(archivo, "%*[^\\n]\\n");
12
13    while (fscanf(archivo, "%[^,],%f,%f,%f\\n",
14                nombre, &nota1, &nota2, &nota3) == 4) {
15        promedio = (nota1 + nota2 + nota3) / 3;
16        printf("%s: %.2f\\n", nombre, promedio);
17    }
18
19    fclose(archivo);
20    return 0;
21 }
```

Compilación / Salida:

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char nombre[50];
6     float nota1, nota2, nota3, promedio;
7
8     archivo = fopen("notas.csv", "r");
9
10    // Saltar encabezado
11    fscanf(archivo, "%*[^\\n]\\n");
12
13    while (fscanf(archivo, "%[^,],%f,%f,%f\\n",
14                nombre, &nota1, &nota2, &nota3) == 4) {
15        promedio = (nota1 + nota2 + nota3) / 3;
16        printf("%s: %.2f\\n", nombre, promedio);
17    }
18
19    fclose(archivo);
20    return 0;
21 }
```

```
C:\Users\Deil Latitude\Downl... x + v
-----
Process exited after 13.24 seconds with return v
Presione una tecla para continuar . . . |
```

Comandos clave para múltiples archivos

Función	Descripción
<code>sprintf(nombre, .arch%d.txt", i)</code>	Genera nombres de archivo dinámicamente.
<code>fopen(nombre, "w")</code>	Escribe (crea o sobrescribe).
<code>fopen(nombre, r")</code>	Lee archivo existente.
<code>fopen(nombre, .a")</code>	Agrega al final sin borrar.
<code>fprintf(archivo, ...)</code>	Escribe con formato.
<code>fscanf(archivo, ...)</code>	Lee con formato.
<code>fgets(linea, tam, archivo)</code>	Lee una línea completa.
<code>fputs(linea, archivo)</code>	Escribe una línea.
<code>fgetc(archivo)</code>	Lee un carácter.
<code>fputc(c, archivo)</code>	Escribe un carácter.

Patrón básico para múltiples archivos

Listing 11: Estructura base para recorrer múltiples archivos

```
1 for (int i = 1; i <= N; i++) {
2     /* 1. Construir nombre */
3     sprintf(nombre, "archivo_%d.txt", i);
4
5     /* 2. Abrir archivo */
6     archivo = fopen(nombre, "modo");
7
8     /* 3. Leer o escribir */
9     // fscanf(...) o fprintf(...)
10
11    /* 4. Cerrar archivo */
12    fclose(archivo);
13 }
```

Consejos prácticos

1. Siempre verificar si `fopen()` retorna `NULL`.
2. Cerrar archivos inmediatamente después de usarlos.
3. Usar nombres descriptivos para los archivos.

4. Modo `.a` para agregar sin borrar contenido previo.
5. Usar bucles para procesar múltiples archivos similares.